

Object_Detection

2D

1. Indirect methods (two-stage): Faster R-CNN
2. Direct methods (one-stage):
 1. anchor based: YOLO, RetinaNet
 2. anchor-free/one anchor: , CenterNet, FCOS
3. Faster R-CNN
 1. Faster RCNN uses VGG16 backbone.
 2. Faster RCNN(RPN + Fast RCNN)
 1. Insert a Region Proposal Network (RPN) after the last convolutional layer -> using GPU!
 2. RPN trained to produce region proposals directly; no need for external region proposals.
 3. after RPN, use ROI pooling and an upstream classifier and bbox regressor just like Fast RCNN.
 3. ROI pooling is `tf.image.crop_and_resize`
 1. Extracts crops from the input image tensor and resizes them using bilinear sampling or nearest neighbor sampling (possibly with aspect ratio change) to a common output size specified by `crop_size`. This is more general than the `crop_to_bounding_box` op which extracts a fixed size slice from the input image and does not allow resizing or aspect ratio change. The crops occur first and then the resize.
 2. Returns a tensor with crops from the input image at positions defined at the bounding box locations in `boxes`. The cropped boxes are all resized (with bilinear or nearest neighbor interpolation) to a fixed `size = [crop_height, crop_width]`. The result is a 4-D tensor `[num_boxes, crop_height, crop_width, depth]`. The resizing is corner aligned. In particular, if `boxes = [[0, 0, 1, 1]]`, the method will give identical results to using `tf.compat.v1.image.resize_bilinear()` or `tf.compat.v1.image.resize_nearest_neighbor()` (depends on the method argument) with `align_corners=True`.
 3. ROI pooling is just a special case of the SPP layer with one pyramid level
 1. ROI in conv feature map: 21x14->3x2 max pooling with stride(3, 2)->output: 7x7
 2. ROI in conv feature map: 35x42->5x6 max pooling with stride(5, 6)->output: 7x7
 3. Related: see how kernel size and stride are computed dynamically in SPPNet in [Convolution](#)
 4. ROI pooling vs ROI align
 1. as part of quantization in ROI pooling it losses a lot of data in the process.
 2. ROI align does not use quantization, but instead it uses bilinear sampling.
 3. after that apply max pooling or avg pooling.
 4. Code: [MSDN/faster_rcnn/roi_pooling/modules/roi_pool_py.py](#)

```
import torch
import torch.nn as nn
from torch.autograd import Variable
import numpy as np

class RoIPool(nn.Module):
    def __init__(self, pooled_height, pooled_width, spatial_scale):
        super(RoIPool, self).__init__()
        self.pooled_width = int(pooled_width)
        self.pooled_height = int(pooled_height)
        self.spatial_scale = float(spatial_scale)

    def forward(self, features, rois):
        batch_size, num_channels, data_height, data_width = features.size()
        num_rois = rois.size()[0]
        outputs = Variable(torch.zeros(num_rois, num_channels, self.pooled_height,
        self.pooled_width)).cuda()

        for roi_ind, roi in enumerate(rois):
            batch_ind = int(roi[0].data[0])
            roi_start_w, roi_start_h, roi_end_w, roi_end_h = np.round(
                roi[1:].data.cpu().numpy() * self.spatial_scale).astype(int)
            roi_width = max(roi_end_w - roi_start_w + 1, 1)
            roi_height = max(roi_end_h - roi_start_h + 1, 1)
            bin_size_w = float(roi_width) / float(self.pooled_width)
            bin_size_h = float(roi_height) / float(self.pooled_height)

            for ph in range(self.pooled_height):
                hstart = int(np.floor(ph * bin_size_h))
                hend = int(np.ceil((ph + 1) * bin_size_h))
                hstart = min(data_height, max(0, hstart + roi_start_h))
                hend = min(data_height, max(0, hend + roi_start_h))
```

```

        for pw in range(self.pooled_width):
            wstart = int(np.floor(pw * bin_size_w))
            wend = int(np.ceil((pw + 1) * bin_size_w))
            wstart = min(data_width, max(0, wstart + roi_start_w))
            wend = min(data_width, max(0, wend + roi_start_w))

            is_empty = (hend <= hstart) or (wend <= wstart)
            if is_empty:
                outputs[roi_ind, :, ph, pw] = 0
            else:
                data = features[batch_ind]
                outputs[roi_ind, :, ph, pw] = torch.max(
                    torch.max(data[:, hstart:hend, wstart:wend], 1)[0], 2)[0].view(-1)

    return outputs

```

5. Code: [MSDN/faster_rcnn/roi_pooling/functions/roi_pool.py](#)

```

import torch
from torch.autograd import Function
from .._ext import roi_pooling

class RoIPoolFunction(Function):
    def __init__(self, pooled_height, pooled_width, spatial_scale):
        self.pooled_width = int(pooled_width)
        self.pooled_height = int(pooled_height)
        self.spatial_scale = float(spatial_scale)
        self.output = None
        self.argmax = None
        self.rois = None
        self.feature_size = None

    def forward(self, features, rois):
        batch_size, num_channels, data_height, data_width = features.size()
        num_rois = rois.size()[0]
        output = torch.zeros(num_rois, num_channels, self.pooled_height, self.pooled_width)
        argmax = torch.IntTensor(num_rois, num_channels, self.pooled_height, self.pooled_width).zero_()

        if not features.is_cuda:
            assert False, 'feature not in CUDA'
            _features = features.permute(0, 2, 3, 1)
            roi_pooling.roi_pooling_forward(self.pooled_height, self.pooled_width, self.spatial_scale,
                                           _features, rois, output)

            # output = output.cuda()
        else:
            output = output.cuda()
            argmax = argmax.cuda()
            roi_pooling.roi_pooling_forward_cuda(self.pooled_height, self.pooled_width, self.spatial_scale,
                                                features, rois, output, argmax)

            self.output = output
            self.argmax = argmax
            self.rois = rois
            self.feature_size = features.size()

        return output

    def backward(self, grad_output):
        # TODO: roi_pooling backward

        assert(self.feature_size is not None and grad_output.is_cuda)

        batch_size, num_channels, data_height, data_width = self.feature_size

        grad_input = torch.zeros(batch_size, num_channels, data_height, data_width).cuda()
        roi_pooling.roi_pooling_backward_cuda(self.pooled_height, self.pooled_width, self.spatial_scale,
                                             grad_output, self.rois, grad_input, self.argmax)

        # print grad_input

        return grad_input, None

```

6. Code: [MSDN/faster_rcnn/roi_pooling/modules/roi_pool.py](#)

```

from torch.nn.modules.module import Module
from ..functions.roi_pool import RoIPoolFunction

class RoIPool(Module):
    def __init__(self, pooled_height, pooled_width, spatial_scale):
        super(RoIPool, self).__init__()

        self.pooled_width = int(pooled_width)
        self.pooled_height = int(pooled_height)
        self.spatial_scale = float(spatial_scale)

    def forward(self, features, rois):
        return RoIPoolFunction(self.pooled_height, self.pooled_width, self.spatial_scale)(features, rois)

```

1. RPN return **trainable ROIs**.
2. slide a small window on the feature map
3. build a small network for
 1. classifying object or not-object
 2. regressing bbox locations
4. position of the sliding window provides localization information with reference to the image
5. box regression provides finer localization information reference to this sliding window
6. *uses k anchor boxes at each location*
7. *anchors are translation invariant: use the same ones at every location.*
8. *regression gives offsets from anchor boxes.*
9. *classification gives the probability that each (regressed) anchor shows an object.*
10. fully convolutional network
 1. intermediate layer: 256 or 512, 3x3 filter, stride 1, padding 1
 2. cls layer: 18(9x2), 1x1 filter, stride 1, padding 0
 3. reg layer: 36(9x4), 1x1 filter, stride 1, padding 0
11. *anchors as references:*
 1. anchors: bbox priors
 2. multi-scale/size anchors
 1. multiple anchors are used at each position: 3 scale (128x128, 256x256, 512x512) and 3 aspect ratios (2:1, 1:1, 1:2) yield 9 anchor combinations
 3. each anchor has its own prediction function
 4. single-scale features, multi-scale predictions
 5. An anchor is positive if:
 1. The anchor/anchors with the highest IoU overlap with a ground-truth box.
 2. an anchor that has an IoU overlap higher than 0.7 with any ground-truth box.
 6. negative if its IoU ratio is lower than 0.3 for all ground-truth boxes.
7. RPN loss function:
 1. $L(p_i, t_i) = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* L_{reg}(t_i, t_i^*)$
 2. the first term is classification loss, the second term is regression loss.
 3. the regression loss is activated only for positive anchors.
 4. i = anchor index in minibatch
 5. L_{cls} is Log loss.
 6. L_{reg} is smooth L1 loss.
 7. p_i is the predicted probability of being an object for anchor i .
 8. t_i is the coordinates of the predicted bbox for anchor i .
 9. t_i^* is the True box coordinates.
 10. p_i^* is the ground truth objectness label.
 11. In practice $\lambda = 10$, so that both terms are roughly equally balanced.
 12. N_{cls} : number of anchors in minibatch ~256.
 13. N_{reg} : number of anchor locations (~2400).
8. 4-step alternating training for FasterRCNN
 1. Step 1: Train RPN with the backbone pretrained on ImageNet.
 2. Step 2: Train Fast RCNN using the proposals generated by the step-1 RPN (At this point the two networks do not share conv layers).
 3. Step 3: Use the backbone from step-2 for RPN training, which will be fixed (not-trainable), and fine-tune the RPN (Now the two networks share conv layers).
 4. Step 4: keeping the backbone from step-2 fixed, fine-tune Fast RCNN.
9. let M0 be an ImageNet pre-trained network
 1. train_rpn(M0) $\rightarrow$ M1
 1. Train an RPN initialized from M0, get M1
 2. generate proposals(M1) $\rightarrow$ P1
 1. Generate training proposals P1 using RPN M1
 3. train_fast_rcnn(M0, P1) $\rightarrow$ M2
 1. Train Fast RCNN M2 on P1 initialized from M0

4. train_rpn_frozen_conv(M2) → M3
 1. Train RPN M3 from M2 *without* changing conv layers
5. generate_proposals(M3) → P2
6. train_fast_rcnn_frozen_conv(M3, P2) → M4
 1. Conv layers are shared with RPN M3
7. return add_rpn_layers(M4, M3.RPN)
 1. Add M3's RPN layers to Fast RCNN M4
12. k=9, number of anchor boxes per feature point in the feature map.
13. number of anchors=9 x 16 x 16
14. (2x9)x16x16 means for each anchor box, we predict 2 cls, obj/no obj. or foreground/background can also use k and sigmoid (Related: see sigmoid+bceloss vs softmax+crossentropy in [Neural Networks](#)), instead of here we're using $2k$ and softmax activation, the author said for simplicity.
15. (4x9)x16x16 for 4 bounding box values.
16. There are 9 anchors generated by sliding window, the sliding window here is done using convolution.
17. Note: For convolutional feature map of size W x H, there are W x H x k anchors in total.
18. Code: [simple-faster-rcnn-pytorch/model/region_proposal_network.py](#)

```
import numpy as np
from torch.nn import functional as F
import torch as t
from torch import nn

from model.utils.bbox_tools import generate_anchor_base
from model.utils.creator_tool import ProposalCreator

class RegionProposalNetwork(nn.Module):
    """Region Proposal Network introduced in Faster R-CNN.

    This is Region Proposal Network introduced in Faster R-CNN [#]_.
    This takes features extracted from images and propose
    class agnostic bounding boxes around "objects".

    .. [#] Shaoqing Ren, Kaiming He, Ross Girshick, Jian Sun. \
    Faster R-CNN: Towards Real-Time Object Detection with \
    Region Proposal Networks. NIPS 2015.

    Args:
        in_channels (int): The channel size of input.
        mid_channels (int): The channel size of the intermediate tensor.
        ratios (list of floats): This is ratios of width to height of
            the anchors.
        anchor_scales (list of numbers): This is areas of anchors.
            Those areas will be the product of the square of an element in
            :obj:`anchor_scales` and the original area of the reference
            window.
        feat_stride (int): Stride size after extracting features from an
            image.
        initialW (callable): Initial weight value. If :obj:`None` then this
            function uses Gaussian distribution scaled by 0.1 to
            initialize weight.
            May also be a callable that takes an array and edits its values.
        proposal_creator_params (dict): Key valued paramters for
            :class:`model.utils.creator_tools.ProposalCreator`.

    .. seealso::
        :class:`~model.utils.creator_tools.ProposalCreator`

    """
    def __init__(
        self, in_channels=512, mid_channels=512, ratios=[0.5, 1, 2],
        anchor_scales=[8, 16, 32], feat_stride=16,
        proposal_creator_params=dict(),
    ):
        super(RegionProposalNetwork, self).__init__()
        self.anchor_base = generate_anchor_base(
            anchor_scales=anchor_scales, ratios=ratios)
        self.feat_stride = feat_stride
        self.proposal_layer = ProposalCreator(self, **proposal_creator_params)
        n_anchor = self.anchor_base.shape[0]
        self.conv1 = nn.Conv2d(in_channels, mid_channels, 3, 1, 1)
        self.score = nn.Conv2d(mid_channels, n_anchor * 2, 1, 1, 0)
        self.loc = nn.Conv2d(mid_channels, n_anchor * 4, 1, 1, 0)
        normal_init(self.conv1, 0, 0.01)
        normal_init(self.score, 0, 0.01)
        normal_init(self.loc, 0, 0.01)

    def forward(self, x, img_size, scale=1.):
        """Forward Region Proposal Network.
```

Here are notations.

- * :math:`N` is batch size.
- * :math:`C` channel size of the input.
- * :math:`H` and :math:`W` are height and width of the input feature.
- * :math:`A` is number of anchors assigned to each pixel.

Args:

- x (~torch.autograd.Variable): The Features extracted from images.
Its shape is :math:`(N, C, H, W)`.
- img_size (tuple of ints): A tuple :obj:`height, width`,
which contains image size after scaling.
- scale (float): The amount of scaling done to the input images after
reading them from files.

Returns:

(~torch.autograd.Variable, ~torch.autograd.Variable, array, array, array):

This is a tuple of five following values.

- * **rpn_locs**: Predicted bounding box offsets and scales for \
anchors. Its shape is :math:`(N, H W A, 4)`.
- * **rpn_scores**: Predicted foreground scores for \
anchors. Its shape is :math:`(N, H W A, 2)`.
- * **rois**: A bounding box array containing coordinates of \
proposal boxes. This is a concatenation of bounding box \
arrays from multiple images in the batch. \
Its shape is :math:`(R', 4)` . Given :math:`R\_i` predicted \
bounding boxes from the :math:`i` th image, \
:math:`R' = \sum\_{i=1}^N R\_i`.
- * **roi_indices**: An array containing indices of images to \
which RoIs correspond to. Its shape is :math:`(R',)`.
- * **anchor**: Coordinates of enumerated shifted anchors. \
Its shape is :math:`(H W A, 4)`.

"""

```
n, _, hh, ww = x.shape
anchor = _enumerate_shifted_anchor(
    np.array(self.anchor_base),
    self.feat_stride, hh, ww)

n_anchor = anchor.shape[0] // (hh * ww)
h = F.relu(self.conv1(x))

rpn_locs = self.loc(h)
# UNNOTE: check whether need contiguous
# A: Yes
rpn_locs = rpn_locs.permute(0, 2, 3, 1).contiguous().view(n, -1, 4)
rpn_scores = self.score(h)
rpn_scores = rpn_scores.permute(0, 2, 3, 1).contiguous()
rpn_softmax_scores = F.softmax(rpn_scores.view(n, hh, ww, n_anchor, 2), dim=4)
rpn_fg_scores = rpn_softmax_scores[:, :, :, :, 1].contiguous()
rpn_fg_scores = rpn_fg_scores.view(n, -1)
rpn_scores = rpn_scores.view(n, -1, 2)

rois = list()
roi_indices = list()
for i in range(n):
    roi = self.proposal_layer(
        rpn_locs[i].cpu().data.numpy(),
        rpn_fg_scores[i].cpu().data.numpy(),
        anchor, img_size,
        scale=scale)
    batch_index = i * np.ones((len(roi),), dtype=np.int32)
    rois.append(roi)
    roi_indices.append(batch_index)

rois = np.concatenate(rois, axis=0)
roi_indices = np.concatenate(roi_indices, axis=0)
return rpn_locs, rpn_scores, rois, roi_indices, anchor
```

def _enumerate_shifted_anchor(anchor_base, feat_stride, height, width):

```
# Enumerate all shifted anchors:
#
# add A anchors (1, A, 4) to
# cell K shifts (K, 1, 4) to get
# shift anchors (K, A, 4)
# reshape to (K*A, 4) shifted anchors
# return (K*A, 4)

# !TODO: add support for torch.CudaTensor
# xp = cuda.get_array_module(anchor_base)
# it seems that it can't be boosed using GPU
import numpy as xp
shift_y = xp.arange(0, height * feat_stride, feat_stride)
shift_x = xp.arange(0, width * feat_stride, feat_stride)
shift_x, shift_y = xp.meshgrid(shift_x, shift_y)
shift = xp.stack((shift_y.ravel(), shift_x.ravel(),
```

```

        shift_y.ravel(), shift_x.ravel()), axis=1)

A = anchor_base.shape[0]
K = shift.shape[0]
anchor = anchor_base.reshape((1, A, 4)) + \
        shift.reshape((1, K, 4)).transpose((1, 0, 2))
anchor = anchor.reshape((K * A, 4)).astype(np.float32)
return anchor

def _enumerate_shifted_anchor_torch(anchor_base, feat_stride, height, width):
    # Enumerate all shifted anchors:
    #
    # add A anchors (1, A, 4) to
    # cell K shifts (K, 1, 4) to get
    # shift anchors (K, A, 4)
    # reshape to (K*A, 4) shifted anchors
    # return (K*A, 4)

    # !TODO: add support for torch.CudaTensor
    # xp = cuda.get_array_module(anchor_base)
    import torch as t
    shift_y = t.arange(0, height * feat_stride, feat_stride)
    shift_x = t.arange(0, width * feat_stride, feat_stride)
    shift_x, shift_y = xp.meshgrid(shift_x, shift_y)
    shift = xp.stack((shift_y.ravel(), shift_x.ravel(),
                      shift_y.ravel(), shift_x.ravel()), axis=1)

    A = anchor_base.shape[0]
    K = shift.shape[0]
    anchor = anchor_base.reshape((1, A, 4)) + \
            shift.reshape((1, K, 4)).transpose((1, 0, 2))
    anchor = anchor.reshape((K * A, 4)).astype(np.float32)
    return anchor

def normal_init(m, mean, stddev, truncated=False):
    """
    weight initializer: truncated normal and random normal.
    """
    # x is a parameter
    if truncated:
        m.weight.data.normal_().fmod_(2).mul_(stddev).add_(mean) # not a perfect approximation
    else:
        m.weight.data.normal_(mean, stddev)
        m.bias.data.zero_()

```

19. Function calls;

1. lines copied from [MSDN/faster_rcnn/MSDN.py](#)

```

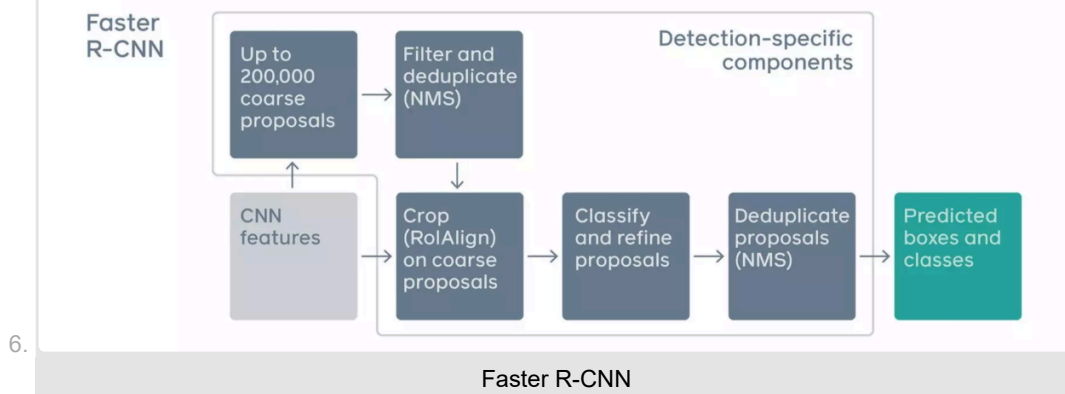
from RPN import RPN
from roi_pooling.modules.roi_pool import RoIPool

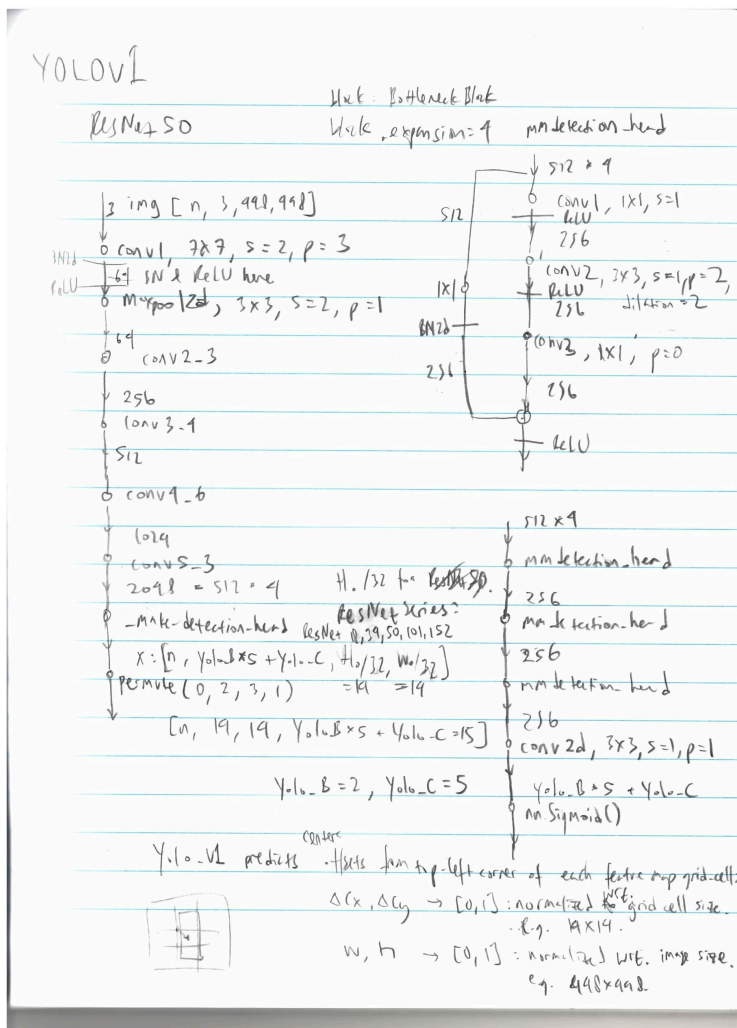
# Initialization
self.rpn = RPN(use_kmeans_anchors)
self.roi_pool_object = RoIPool(7, 7, 1.0/16)
self.roi_pool_phrase = RoIPool(7, 7, 1.0/16)
self.roi_pool_region = RoIPool(7, 7, 1.0/16)

# Function calls
features, object_rois, region_rois = self.rpn(im_data, im_info, gt_objects, gt_regions)
# roi pool
pooled_object_features = self.roi_pool_object(features, object_rois)
pooled_phrase_features = self.roi_pool_phrase(features, phrase_rois)
pooled_region_features = self.roi_pool_region(features, region_rois)

```

Streamlined Detection Pipeline





- 1.
2. YOLO treats object detection as a **regression problem**. Specifically, we encode the gt bbox centers as offsets from the top-left corner of each cell in the feature map into 0-1 range (as we've known neural networks like to work with values in the range -1 to 1 or 0 to 1). The network will then learn to regress these offsets which is in the range 0-1.
3. S: YOLO grid num,
4. B: YOLO box num,
5. C: detection class num.
6. The image is resized to (image_size, image_size) before input to the backbone. For example image_size=224 or 448 are commonly used (224 or 448 for Resnet backbone, that outputs 7x7 or 14x14 feature map).
7. In original yolov1 input size is 448 x 448, **downsampling factor** is 64, output grid size is 7 x 7.
8. In yolov2, one max-pooling layer is removed. Input size is 448 x 448, **downsampling factor** is 32, output grid size is 14 x 14.
9. The TLWH box format from COCO is converted to $x_1y_1x_2y_2$ format and divided by (w,h,w,h).
10. The box encoder purpose is to provide (image, target) pair for training. In box encoder the boxes in $x_1y_1x_2y_2$ format is converted to **YOLO format**, x_cy_cwh format. The offset from the center of each bbox to the top-left corner of each grid cell which is Δ_{xy} is computed by :
 1. For instance, if S=num_cell=7 (or 14), then cell_size = 1/7.
 2. To convert to grid cell coordinate divide x_cy_c by cell_size. To convert back to image coordinate multiply by cell_size.
 3. In grid cell coordinate, $ij = \text{floor}(cxcy_sample/cell_size)$, top-left corner (in image coordinate): $xy = ij * cell_size$. The offset in grid cell coordinate is $\Delta_{xy} = (cxcy_sample - xy)/cell_size$. Now, Δ_{xy} is in the range [0,1] in grid cell coordinate and is to be regressed by the network.
 4. essentially, same as getting the offset in CenterNet: let $cell_size = c$, $(p_x - \lfloor \frac{p_x}{c} \rfloor * c)/c = \frac{p_x}{c} - \lfloor \frac{p_x}{c} \rfloor$, same with y.
11. YOLO outputs a feature map tensor of size: prediction: shape (S, S, B * 5 + C). B is the number of candidate boxes.
 Note: **there is no concept of anchor boxes in YOLOv1**.
12. why 5? because the box is of the format $[\delta_x, \delta_y, w, h, conf]$ plus C possible classes for the one box predicted per grid cell.
13. In YOLOv1 it is assumed that there is only one true box per grid cell. Therefore, to match the size of the output tensor (for computing loss). In the third axis of size $B * 5 + C$ only one ground truth box is used and is repeated B consecutive

times throughout the third axis.

14. **Computing loss:** To recover $x_1y_1x_2y_2$ format to compute IOU, because width and height of box is always in image coordinate it does not need to be multiplied by `cell_size`:

1. $\text{box_xyxy}[:, :2] = \text{box1}[:, :2] * \text{cell_size} - 0.5 * \text{box1}[:, 2:4]$
2. $\text{box_xyxy}[:, 2:4] = \text{box1}[:, :2] * \text{cell_size} + 0.5 * \text{box1}[:, 2:4]$
3. Note: boxes are already reshaped into shape: $[S*S*B, 5]$, 4 + 1 for confidence score:

```
box_pred = contain_object_pred[:, :self.B * 5].contiguous().view(-1, 5)
...
box_target = contain_object_target[:, :self.B * 5].contiguous().view(-1, 5)
```

4. Code:

```
for i in range(0, box_target.size(0), self.B):
    box1 = box_pred[i:i + self.B]
    box1_xyxy = torch.zeros(box1.size(), dtype=torch.float32, device='cuda')
    """ from [x_c, y_c, w, h] to [x1, y1, x2, y2] """
    cell_size = 1 / self.S
    box1_xyxy[:, :2] = box1[:, :2] * cell_size - 0.5 * box1[:, 2:4]
    box1_xyxy[:, 2:4] = box1[:, :2] * cell_size + 0.5 * box1[:, 2:4]
    box1_xyxy[:, 4] = box1[:, 4]

    box2 = box_target[i].view(-1, 5)
    box2_xyxy = torch.zeros(box2.size(), dtype=torch.float32, device='cuda')
    box2_xyxy[:, :2] = box2[:, :2] * cell_size - 0.5 * box2[:, 2:4]
    box2_xyxy[:, 2:4] = box2[:, :2] * cell_size + 0.5 * box2[:, 2:4]
    box2_xyxy[:, 4] = box2[:, 4]

    iou_res = self.compute_iou(box1_xyxy[:, :4], box2_xyxy[:, :4])
    max_iou, max_index = torch.max(iou_res, dim=0)
    max_index = max_index.item()

    ...
```

5. This is done so because when combining the two information they should share the same coordinate scale.

6. Note: normally a function to compute IOU expects $x_1y_1x_2y_2$ format.

15. Box width and height are regressed directly as after normalization by image_size they are in the range[0,1] in image coordinates.
16. Bipartite matching: During training, bounding box assignment is done to assign one candidate ground truth box (since there are B copies of the same ground truth box) to B prediction boxes in each grid cell based on their IOUs. Note: not all B predicted boxes may ever get to be assigned during training. Note: This part is replaced with Hungarian algorithm in DETR, which is used during training only.
17. The last activation function in the model is a Sigmoid function.
18. The loss function is in the form:
19. $L = L_{\text{Localisation}} + \lambda_{\text{coobj}} * L_{\text{coobj}} + \lambda_{\text{noobj}} * (L_{\text{noobj}} + L_{\text{coo_no_response}}) + L_{\text{classification}}$
20. What I learned from implementing YOLOv1 from scratch:
1. Training to improve mAP is tough. After 100 epochs the model only achieved ~2% mAP on the CODA dataset.
 2. The idea of using anchor boxes becomes obvious.
 3. Ways to improve mAP include hyperparameter tuning such as grid search using wandb sweeps.
 4. example, box encoder:
 1. box's width and height normalized to the entire image size (this is done after data augmentation, with some operations modifying both the image and bbox sizes i.e. the bbox coordinates, in xyxy format, is normalized by the sizes of the image after data augmentation $\text{bbox} * \text{scale_tensor}$).
 2. center coordinates x, y: normalized wrt each grid cell.
 3. $\text{image.shape}=(h, w, c)=(448, 448, 3)$
10. encoder returns: target.shape: (14, 14, 15)
11. torch's `__getitem__` automatically add batch_size dim.
12. Code:

```
DATASET INITIALIZATION
image_0
__getitem__
h0, w0 : 720 1280
Data augmentation
img, boxes = random_flip(img, boxes)
img, boxes = randomScale(img, boxes)
img = randomBlur(img)
img = RandomBrightness(img)
img = RandomHue(img)
img = RandomSaturation(img)
img, boxes, labels = randomShift(img, boxes, labels)
img, boxes, labels = randomCrop(img, boxes, labels)
```

```

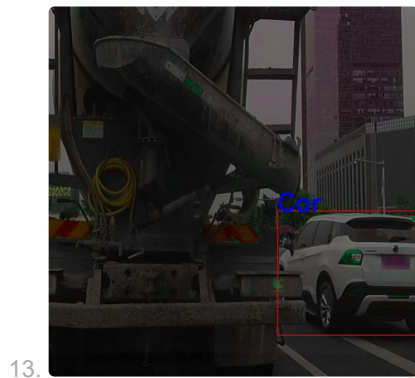
h, w : 654 847
boxes /= torch.tensor([w, h, w, h]).expand_as(boxes)
Transform image
    img = BGR2RGB(img)
    img = subMeanDividedStd(img, self.mean, self.std)
    img = cv2.resize(img, (self.image_size, self.image_size))
transform=[transforms.ToTensor()]
len(boxes) : 1
encoder:
    boxes.shape torch.Size([1, 4])
    labels.shape torch.Size([1])
    boxes tensor([[0.6198, 0.5535, 1.0000, 0.8884]])
    labels tensor([2])
    cell_size 0.07142857142857142
    wh.shape torch.Size([1, 2])
    wh tensor([[0.3802, 0.3349]])
    cxcy.shape torch.Size([1, 2])
    cxcy tensor([[0.8099, 0.7209]])
    target.shape torch.Size([14, 14, 15])
img_0.shape torch.Size([1, 3, 448, 448])
target_0.shape torch.Size([1, 14, 14, 15])
Visualize
    target.shape torch.Size([1, 14, 14, 15])
img_path vis_results/image_0.jpg

```

```

image_1
__getitem__
h0, w0 : 720 1281
Data augmentation
    img, boxes = random_flip(img, boxes)
    img, boxes = randomScale(img, boxes)
    img = randomBlur(img)
    img = RandomBrightness(img)
    img = RandomHue(img)
    img = RandomSaturation(img)
    img, boxes, labels = randomShift(img, boxes, labels)
    img, boxes, labels = randomCrop(img, boxes, labels)
h, w : 714 908
boxes /= torch.tensor([w, h, w, h]).expand_as(boxes)
Transform image
    img = BGR2RGB(img)
    img = subMeanDividedStd(img, self.mean, self.std)
    img = cv2.resize(img, (self.image_size, self.image_size))
transform=[transforms.ToTensor()]
len(boxes) : 7
encoder:
    boxes.shape torch.Size([7, 4])
    labels.shape torch.Size([7])
    boxes tensor([[0.5061, 0.8011, 0.5386, 0.8319],
                  [0.3399, 0.8081, 0.4126, 0.8641],
                  [0.6282, 0.7661, 0.7671, 0.8852],
                  [0.1931, 0.7885, 0.2373, 0.9202],
                  [0.4594, 0.8053, 0.4866, 0.8291],
                  [0.1594, 0.7913, 0.2438, 0.9328],
                  [0.2490, 0.8501, 0.2918, 0.9258]])
    labels tensor([2, 2, 2, 0, 2, 0, 0])
    cell_size 0.07142857142857142
    wh.shape torch.Size([7, 2])
    wh tensor([[0.0325, 0.0308],
               [0.0727, 0.0560],
               [0.1390, 0.1190],
               [0.0442, 0.1317],
               [0.0273, 0.0238],
               [0.0844, 0.1415],
               [0.0429, 0.0756]])
    cxcy.shape torch.Size([7, 2])
    cxcy tensor([[0.5223, 0.8165],
                 [0.3762, 0.8361],
                 [0.6977, 0.8256],
                 [0.2152, 0.8543],
                 [0.4730, 0.8172],
                 [0.2016, 0.8620],
                 [0.2704, 0.8880]])
    target.shape torch.Size([14, 14, 15])
img_1.shape torch.Size([1, 3, 448, 448])
target_1.shape torch.Size([1, 14, 14, 15])
Visualize
    target.shape torch.Size([1, 14, 14, 15])
img_path vis_results/image_1.jpg

```



21. yolov1 box encoder: src/yolo/data/dataset.py: <https://github.com/Liang-ZX/HKU-DASC7606-A1/blob/main/src/data/dataset.py>

```
import os
import json
import argparse

import random
import numpy as np

import torch
import torch.utils.data as data

import cv2

CAR_CLASSES = ['Pedestrian', 'Cyclist', 'Car', 'Truck',
               'Tram']

COLORS = {'Pedestrian': (0, 0, 0),
          'Cyclist': (128, 0, 0),
          'Car': (0, 128, 0),
          'Truck': (128, 128, 0),
          'Tram': (0, 0, 128)}

class Dataset(data.Dataset):
    image_size = 448

    def __init__(self, args, split, transform):
        print('DATASET INITIALIZATION')
        self.args = args
        root = args.dataset_root
        self.root_images = os.path.join(root, split, 'image')
        if split == "train":
            self.train = True
        else:
            self.train = False

        self.transform = transform
        self.f_names, self.bboxes, self.labels = [], [], []
        self.mean = [123.675, 116.280, 103.530] # RGB
        self.std = [58.395, 57.120, 57.375]
        annotation_path = os.path.join(root, 'annotations', 'instance_' + split + '.json')
        annotations = load_json(annotation_path)

        for annotation in annotations['annotations']:
            # if annotation['image_name'] not in self.f_names:
            #     if len(self.f_names) != 0:
            #         self.bboxes.append(torch.Tensor(box))
            #         self.labels.append(torch.LongTensor(label))
            #         # box, label = [], []
            #         self.f_names.append(annotation['image_name'])

            # bbox = annotation['bbox']
            # x1, y1, x2, y2 = float(bbox[0]), float(bbox[1]), float(bbox[0] + bbox[2]), float(bbox[1] + bbox[3])
```

```

        # box.append([x1, y1, x2, y2])
        # label.append(int(annotation['category_id']))
        bboxes = []; labels=[]
        for bbox in annotation['bbox']:
            x1, y1, x2, y2 = float(bbox[0]), float(bbox[1]), float(bbox[0] + bbox[2]), float(bbox[1] +
bbox[3])
            bboxes.append([x1, y1, x2, y2])
        for cat_id in annotation['category_id']:
            labels.append(cat_id)
        if len(bboxes)>0:
            self.f_names.append(annotation['image_name'])
            self.bboxes.append(torch.Tensor(bboxes))
            self.labels.append(torch.LongTensor(labels))

        # self.bboxes.append(torch.Tensor(box))
        # self.labels.append(torch.LongTensor(label))

        self.num_samples = len(self.bboxes)

    def __getitem__(self, idx):
        print("__getitem__")
        f_name = self.f_names[idx]
        img = cv2.imread(os.path.join(self.root_images, f_name))
        boxes = self.bboxes[idx].clone()
        labels = self.labels[idx].clone()

        h0, w0, _ = img.shape
        print(" h0, w0 :", h0, w0)

        # Data augmentation
        print(" Data augmentation")
        print(" img, boxes = random_flip(img, boxes)")
        print(" img, boxes = randomScale(img, boxes)")
        print(" img = randomBlur(img)")
        print(" img = RandomBrightness(img)")
        print(" img = RandomHue(img)")
        print(" img = RandomSaturation(img)")
        print(" img, boxes, labels = randomShift(img, boxes, labels)")
        print(" img, boxes, labels = randomCrop(img, boxes, labels)")
        if self.train and boxes.ndim > 1:
            # img = self.random_bright(img)
            img, boxes = random_flip(img, boxes)
            img, boxes = randomScale(img, boxes)
            img = randomBlur(img)
            img = RandomBrightness(img)
            img = RandomHue(img)
            img = RandomSaturation(img)
            img, boxes, labels = randomShift(img, boxes, labels)
            img, boxes, labels = randomCrop(img, boxes, labels)

        h, w, _ = img.shape
        print(" h, w :", h, w)
        boxes /= torch.tensor([w, h, w, h]).expand_as(boxes)
        print(" boxes /= torch.tensor([w, h, w, h]).expand_as(boxes)")

        # Transform image
        print(" Transform image")
        print(" img = BGR2RGB(img)")
        print(" img = subMeanDividedStd(img, self.mean, self.std)")
        print(" img = cv2.resize(img, (self.image_size, self.image_size))")
        img = BGR2RGB(img)
        img = subMeanDividedStd(img, self.mean, self.std)
        img = cv2.resize(img, (self.image_size, self.image_size))

        for t in self.transform:
            img = t(img)
        print(" transform=[transforms.ToTensor()]")

        if len(boxes) == 0:
            S, B, C = self.args.yolo_S, self.args.yolo_B, self.args.yolo_C
            grid_num = S
            target = torch.zeros((grid_num, grid_num, B * 5 + C))
            return img, target

        print(" len(boxes) :", len(boxes))
        target = self.encoder(boxes, labels) # S*S*(B*5+C)
        return img, target

    def __len__(self):
        return self.num_samples

    def encoder(self, boxes, labels):
        print(" encoder:")
        print(" boxes.shape", boxes.shape)
        print(" labels.shape", labels.shape)
        print(" boxes", boxes)
        print(" labels", labels)
        S, B, C = self.args.yolo_S, self.args.yolo_B, self.args.yolo_C
        grid_num = S
        target = torch.zeros((grid_num, grid_num, B * 5 + C))

```

```

cell_size = 1.0 / grid_num
print("    cell_size", cell_size)
wh = boxes[:, 2:] - boxes[:, :2]
print("    wh.shape", wh.shape)
print("    wh", wh)
cxcy = (boxes[:, 2:] + boxes[:, :2]) / 2
print("    cxcy.shape", cxcy.shape)
print("    cxcy", cxcy)
for i in range(cxcy.size(0)):
    cxcy_sample = cxcy[i]
    ij = (cxcy_sample / cell_size).floor()
    if ij[0] >= grid_num or ij[1] >= grid_num:
        print(f"Out of bounds! {ij}")
        print("cxcy", cxcy)
        print("")
        # clamp
        ij[0] = min(max(ij[0], 0), grid_num - 1)
        ij[1] = min(max(ij[1], 0), grid_num - 1)
    for kk in range(B):
        target[int(ij[1]), int(ij[0]), kk*5 + 4] = 1
    target[int(ij[1]), int(ij[0]), int(B*5 + labels[i])] = 1
    # top-left corner of grid cell containing the gt box in image coordinates
    xy = ij * cell_size
    # Distance from center of gt box to the top-left corner
    delta_xy = cxcy_sample - xy
    # Convert to cell coordinates
    delta_xy = delta_xy / cell_size
    # delta_xy = cxcy_sample - xy
    # Because Yolov1 assume only a gt box per grid cell,
    # fill target tensor with a duplicated gt box
    for kk in range(B):
        target[int(ij[1]), int(ij[0]), kk*5 : kk*5 + 2] = delta_xy
        target[int(ij[1]), int(ij[0]), kk*5 + 2 : kk*5 + 4] = wh[i]
    print("    target.shape", target.shape)
    # print("target", target)
    # print("\n")
    return target

def num_classes(self):
    return len(CAR_CLASSES)

def BGR2RGB(img):
    return cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

def BGR2HSV(img):
    return cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

def HSV2BGR(img):
    return cv2.cvtColor(img, cv2.COLOR_HSV2BGR)

def RandomBrightness(bgr):
    if random.random() < 0.5:
        hsv = BGR2HSV(bgr)
        h, s, v = cv2.split(hsv)
        adjust = random.choice([0.5, 1.5])
        v = v * adjust
        v = np.clip(v, 0, 255).astype(hsv.dtype)
        hsv = cv2.merge((h, s, v))
        bgr = HSV2BGR(hsv)
    return bgr

def RandomSaturation(bgr):
    if random.random() < 0.5:
        hsv = BGR2HSV(bgr)
        h, s, v = cv2.split(hsv)
        adjust = random.choice([0.5, 1.5])
        s = s * adjust
        s = np.clip(s, 0, 255).astype(hsv.dtype)
        hsv = cv2.merge((h, s, v))
        bgr = HSV2BGR(hsv)
    return bgr

def RandomHue(bgr):
    if random.random() < 0.5:
        hsv = BGR2HSV(bgr)
        h, s, v = cv2.split(hsv)
        adjust = random.choice([0.5, 1.5])
        h = h * adjust
        h = np.clip(h, 0, 255).astype(hsv.dtype)
        hsv = cv2.merge((h, s, v))
        bgr = HSV2BGR(hsv)
    return bgr

def randomBlur(bgr):

```

```

    if random.random() < 0.5:
        bgr = cv2.blur(bgr, (5, 5))
    return bgr

def randomShift(bgr, boxes, labels):
    # print("boxes", boxes)
    # print("boxes.shape", boxes.shape)
    # if boxes.ndim > 1:
    center = (boxes[:, 2:] + boxes[:, :2]) / 2
    if random.random() < 0.5:
        height, width, c = bgr.shape
        after_shfit_image = np.zeros((height, width, c), dtype=bgr.dtype)
        after_shfit_image[:, :, :] = (104, 117, 123) # bgr
        shift_x = random.uniform(-width * 0.2, width * 0.2)
        shift_y = random.uniform(-height * 0.2, height * 0.2)

        if shift_x >= 0 and shift_y >= 0:
            after_shfit_image[int(shift_y):, int(shift_x):, :] = bgr[:height - int(shift_y), :width -
int(shift_x),
                                :]
        elif shift_x >= 0 and shift_y < 0:
            after_shfit_image[:height + int(shift_y), int(shift_x):, :] = bgr[-int(shift_y):, :width -
int(shift_x),
                                :]
        elif shift_x < 0 and shift_y >= 0:
            after_shfit_image[int(shift_y):, :width + int(shift_x), :] = bgr[:height - int(shift_y), -
int(shift_x):,
                                :]
        elif shift_x < 0 and shift_y < 0:
            after_shfit_image[:height + int(shift_y), :width + int(shift_x), :] = bgr[-int(shift_y):,
-int(shift_x):, :]

        shift_xy = torch.FloatTensor([[int(shift_x), int(shift_y)]]).expand_as(center)
        center = center + shift_xy
        mask1 = (center[:, 0] > 0) & (center[:, 0] < width)
        mask2 = (center[:, 1] > 0) & (center[:, 1] < height)
        mask = (mask1 & mask2).view(-1, 1)
        boxes_in = boxes[mask.expand_as(boxes)].view(-1, 4)
        if len(boxes_in) == 0:
            return bgr, boxes, labels
        box_shift = torch.FloatTensor([[int(shift_x), int(shift_y), int(shift_x), int(shift_y)]]).expand_as(
            boxes_in)
        boxes_in = boxes_in + box_shift
        labels_in = labels[mask.view(-1)]
        return after_shfit_image, boxes_in, labels_in
    return bgr, boxes, labels

def randomScale(bgr, boxes):
    if random.random() < 0.5:
        scale = random.uniform(0.8, 1.2)
        height, width, c = bgr.shape
        bgr = cv2.resize(bgr, (int(width * scale), height))
        scale_tensor = torch.FloatTensor([[scale, 1, scale, 1]]).expand_as(boxes)
        boxes = boxes * scale_tensor
    return bgr, boxes

def randomCrop(bgr, boxes, labels):
    if random.random() < 0.5:
        center = (boxes[:, 2:] + boxes[:, :2]) / 2
        height, width, c = bgr.shape
        h = random.uniform(0.6 * height, height)
        w = random.uniform(0.6 * width, width)
        x = random.uniform(0, width - w)
        y = random.uniform(0, height - h)
        x, y, h, w = int(x), int(y), int(h), int(w)

        center = center - torch.FloatTensor([[x, y]]).expand_as(center)
        mask1 = (center[:, 0] > 0) & (center[:, 0] < w)
        mask2 = (center[:, 1] > 0) & (center[:, 1] < h)
        mask = (mask1 & mask2).view(-1, 1)

        boxes_in = boxes[mask.expand_as(boxes)].view(-1, 4)
        if len(boxes_in) == 0:
            return bgr, boxes, labels
        box_shift = torch.FloatTensor([[x, y, x, y]]).expand_as(boxes_in)

        boxes_in = boxes_in - box_shift
        boxes_in[:, 0] = boxes_in[:, 0].clamp_(min=0, max=w)
        boxes_in[:, 2] = boxes_in[:, 2].clamp_(min=0, max=w)
        boxes_in[:, 1] = boxes_in[:, 1].clamp_(min=0, max=h)
        boxes_in[:, 3] = boxes_in[:, 3].clamp_(min=0, max=h)

        labels_in = labels[mask.view(-1)]
        img_cropped = bgr[y:y + h, x:x + w, :]
        return img_cropped, boxes_in, labels_in
    return bgr, boxes, labels

```

```

def subMean(bgr, mean):
    mean = np.array(mean, dtype=np.float32)
    bgr = bgr - mean
    return bgr

def subMeanDividedStd(rgb, mean, std):
    mean = np.array(mean, dtype=np.float32)
    std = np.array(std, dtype=np.float32)
    rgb = (rgb - mean) / std
    return rgb

def random_flip(im, boxes):
    if random.random() < 0.5:
        im_lr = np.fliplr(im).copy()
        h, w, _ = im.shape
        xmin = w - boxes[:, 2]
        xmax = w - boxes[:, 0]
        boxes[:, 0] = xmin
        boxes[:, 2] = xmax
        return im_lr, boxes
    return im, boxes

def random_bright(im, delta=16):
    alpha = random.random()
    if alpha > 0.3:
        im = im * alpha + random.randrange(-delta, delta)
        im = im.clip(min=0, max=255).astype(np.uint8)
    return im

def load_json(path):
    with open(path, mode="r") as f:
        data = json.load(f)
    return data

def visualize(img, target, idx):
    mean = np.array([0.485, 0.456, 0.406]) # RGB
    std = np.array([0.229, 0.224, 0.225])
    img = img.detach().cpu().numpy().squeeze(0).transpose(1,2,0)
    img = img * std + mean
    img = (img*255).clip(0, 255).astype(np.uint8)
    img = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
    img_h, img_w, _ = img.shape
    _, S, S, D = target.shape
    print("Visualize")
    print(" target.shape", target.shape)
    B = (D - 5) // 5
    for i in range(S):
        for j in range(S):
            for b in range(B):
                if target[0, i, j, 5*b+4] > 0:
                    gx = j; gy = i
                    cx = target[0, i, j, 5*b]
                    cy = target[0, i, j, 5*b+1]
                    w = target[0, i, j, 5*b+2]
                    h = target[0, i, j, 5*b+3]
                    x1 = (gx + cx)/S - w/2
                    y1 = (gy + cy)/S - h/2
                    x2 = (gx + cx)/S + w/2
                    y2 = (gy + cy)/S + h/2
                    # rescale
                    x1*=img_w
                    y1*=img_h
                    x2*=img_w
                    y2*=img_h
                    cls = int(torch.argmax(target[0, i, j, 5*B:]).detach().cpu().numpy())
                    label = CAR_CLASSES[cls]
                    cv2.rectangle(img, (int(x1), int(y1)), (int(x2), int(y2)), (0,0,255), 1)
                    cv2.putText(img, label, (int(x1), int(y1)), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0), 2)

    cv2.imshow('window', img)
    # save_path = os.path.join("./vis_results")
    img_path = os.path.join("vis_results", f"image_{idx}.jpg")
    print("img_path", img_path)
    cv2.imwrite(img_path, img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
    # pass

def main():
    from torch.utils.data import DataLoader
    import torchvision.transforms as transforms
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_root', default='./coda_small', type=str, help='dataset_root')

```

```

parser.add_argument('--yolo_S', default=14, type=int, help='YOLO grid num')
parser.add_argument('--yolo_B', default=2, type=int, help='YOLO box num')
parser.add_argument('--yolo_C', default=5, type=int, help='detection class num')
args = parser.parse_args()
train_dataset = Dataset(args, split='train',
                        transform=[transforms.ToTensor()])

# train_loader = DataLoader(train_dataset, batch_size=1, shuffle=False, num_workers=os.cpu_count() - 2)
train_loader = DataLoader(train_dataset, batch_size=1, shuffle=False, num_workers=0)
train_iter = iter(train_loader)
ctr = 0
for i in range(5):
    print(f"image_{i}")
    img, target = next(train_iter)
    print(f'img_{i}.shape', img.shape)
    print(f'target_{i}.shape', target.shape)
    visualize(img, target, i)
    ctr+=1
    if ctr == 2:
        break
    print("\n")

if __name__ == '__main__':
    main() # for debug

```

22. yolov2 box encoder: <https://github.com/kuangliu/pytorch-yolov2/blob/master/encoder.py>

```

'''Encode target locations and class labels.'''
import math
import torch

from utils import box_iou, box_nms, meshgrid, softmax

class DataEncoder:
    def __init__(self):
        self.anchors = [(1.3221,1.73145),(3.19275,4.00944),(5.05587,8.09892),(9.47112,4.84053),(11.2364,10.0071)]

    def encode(self, boxes, labels, input_size):
        '''Encode target bounding boxes and class labels into YOLOv2 format.

        Args:
            boxes: (tensor) bounding boxes of (xmin,ymin,xmax,ymax) in range [0,1], sized [#obj, 4].
            labels: (tensor) object class labels, sized [#obj,].
            input_size: (int) model input size.

        Returns:
            loc_targets: (tensor) encoded bounding boxes, sized [5,4,fmsize,fmsize].
            cls_targets: (tensor) encoded class labels, sized [5,20,fmsize,fmsize].
            box_targets: (tensor) truth boxes, sized [#obj,4].
        '''
        num_boxes = len(boxes)
        # input_size -> fmsize
        # 320->10, 352->11, 384->12, 416->13, ..., 608->19
        fmsize = (input_size - 320) / 32 + 10
        grid_size = input_size / fmsize

        # 5 is the number of anchor boxes variations
        boxes *= input_size # scale [0,1] -> [0,input_size]
        bx = (boxes[:,0] + boxes[:,2]) * 0.5 / grid_size # values \in [0, fmsize]
        by = (boxes[:,1] + boxes[:,3]) * 0.5 / grid_size # values \in [0, fmsize]
        bw = (boxes[:,2] - boxes[:,0]) / grid_size # values \in [0, fmsize]
        bh = (boxes[:,3] - boxes[:,1]) / grid_size # values \in [0, fmsize]

        # Offsets from top left corner of each grid cell
        tx = bx - bx.floor()
        ty = by - by.floor()

        # Anchor boxes creation
        xy = meshgrid(fmsize, swap_dims=True) + 0.5 # grid center, shape: [fmsize*fmsize, 2]
        wh = torch.tensor(self.anchors) # [5,2]
        xy = xy.view(fmsize,fmsize,1,2).expand(fmsize,fmsize,5,2)
        wh = wh.view(1,1,5,2).expand(fmsize,fmsize,5,2)
        anchor_boxes = torch.cat([xy-wh/2, xy+wh/2], 3) # shape: [fmsize, fmsize, 5, 4]
        # num_anchors = fmsize * fmsize * 5
        # box_iou((num_anchors, 4), (num_boxes, 4)) -> ious: shape: [num_anchors, num_boxes]
        ious = box_iou(anchor_boxes.view(-1,4), boxes/grid_size) # shape: [fmsize * fmsize * 5, num_boxes]

        ious = ious.view(fmsize,fmsize,5,num_boxes) # shape: [fmsize, fmsize, 5, num_boxes]

        loc_targets = torch.zeros(5,4,fmsize,fmsize) # shape: [5boxes, 4coords, fmsize, fmsize]
        cls_targets = torch.zeros(5,20,fmsize,fmsize) # shape: [5boxes, 20classes, fmsize, fmsize]
        for i in range(num_boxes):
            cx = int(bx[i])
            cy = int(by[i])
            value, max_idx = ious[cy,cx,:,i].max(0, keepdim=True)
            # value: shape: [1]
            # max_idx: shape: [1]

```

```

        j = max_idx[0]

        tw = bw[i] / self.anchors[j][0]
        th = bh[i] / self.anchors[j][1]

        cls_targets[j, labels[i], cy, cx] = 1
        loc_targets[j, :, cy, cx] = torch.tensor([tx[i], ty[i], tw, th])
    return loc_targets, cls_targets, boxes/grid_size

def decode(self, outputs, input_size):
    '''Transform predicted loc/conf back to real bbox locations and class labels.

    Args:
        outputs: (tensor) model outputs, sized [1,125,13,13].
        input_size: (int) model input size.

    Returns:
        boxes: (tensor) bbox locations, sized [#obj, 4].
        labels: (tensor) class labels, sized [#obj, 1].
    '''
    fmsize = outputs.size(2)
    outputs = outputs.view(5, 25, 13, 13)

    # outputs, dim=1 : [loc_xy: 2 values, loc_wh: 2 values,
    #                  iou_preds: 1 value, cls_preds: 20 values], total: 25 values
    # model outputs: [1, 125, 13, 13], why 125?
    # 125 = 5 (anchor_box_variations) * 25 (box_pred)

    loc_xy = outputs[:, :, 2, :, :] # [5, 2, 13, 13]
    grid_xy = meshgrid(fmsize, swap_dims=True).view(fmsize, fmsize, 2).permute(2, 0, 1) # [2, 13, 13]
    box_xy = loc_xy.sigmoid() + grid_xy.expand_as(loc_xy) # [5, 2, 13, 13]

    loc_wh = outputs[:, :, 2:4, :, :] # [5, 2, 13, 13]
    anchor_wh = torch.Tensor(self.anchors).view(5, 2, 1, 1).expand_as(loc_wh) # [5, 2, 13, 13]
    box_wh = anchor_wh * loc_wh.exp() # [5, 2, 13, 13]

    boxes = torch.cat([box_xy - box_wh / 2, box_xy + box_wh / 2], 1) # [5, 4, 13, 13]
    boxes = boxes.permute(0, 2, 3, 1).contiguous().view(-1, 4) # [845, 4]

    iou_preds = outputs[:, 4, :, :].sigmoid() # [5, 13, 13]
    cls_preds = outputs[:, 5, :, :].sigmoid() # [5, 20, 13, 13]
    cls_preds = cls_preds.permute(0, 2, 3, 1).contiguous().view(-1, 20)
    cls_preds = softmax(cls_preds) # [5*13*13, 20]

    score = cls_preds * iou_preds.view(-1).unsqueeze(1).expand_as(cls_preds) # [5*13*13, 20]
    score = score.max(1)[0].view(-1) # [5*13*13,]
    print(iou_preds.max())
    print(cls_preds.max())
    print(score.max())

    ids = (score > 0.5).nonzero().squeeze()
    keep = box_nms(boxes[ids], score[ids])
    return boxes[ids][keep] / fmsize

```

23. yolov2 dataset.py

```

'''Load image/class/box from a annotation file.

The list file is like:

... img.jpg width height xmin ymin xmax ymax label xmin ymin xmax ymax label ...

from __future__ import print_function

import os
import sys
import os.path

import random
import numpy as np

import torch
import torch.utils.data as data
import torchvision.transforms as transforms

from utils import box_iou
from encoder import DataEncoder
from PIL import Image, ImageOps

class ListDataset(data.Dataset):
    input_sizes = [320 + 32*i for i in range(10)]

    def __init__(self, root, list_file, train, transform):
        ...

    Args:
        root: (str) directory to images.
        list_file: (str) path to index file.
        train: (boolean) train or test.
        transform: ([transforms]) image transforms.
    '''

```

```

self.root = root
self.train = train
self.transform = transform

self.fnames = []
self.bboxes = []
self.labels = []

self.data_encoder = DataEncoder()

with open(list_file) as f:
    lines = f.readlines()
    self.num_samples = len(lines)

for line in lines:
    splited = line.strip().split()
    self.fnames.append(splited[0])
    num_bboxes = (len(splited) - 3) // 5
    box = []
    label = []
    for i in range(num_bboxes):
        xmin = splited[3+5*i]
        ymin = splited[4+5*i]
        xmax = splited[5+5*i]
        ymax = splited[6+5*i]
        c = splited[7+5*i]
        box.append([float(xmin),float(ymin),float(xmax),float(ymax)])
        label.append(int(c))
    self.bboxes.append(torch.Tensor(box))
    self.labels.append(torch.LongTensor(label))

def __getitem__(self, idx):
    '''Load a image, and encode its bbox locations and class labels.

    Args:
        idx: (int) image index.

    Returns:
        img: (tensor) image tensor.
        loc_targets: (tensor) location targets.
        cls_targets: (tensor) class label targets.
        box_targets: (tensor) truth box targets.
        ...

    # Load image and bbox locations.
    fname = self.fnames[idx]
    img = Image.open(os.path.join(self.root, fname))
    bboxes = self.bboxes[idx].clone()
    labels = self.labels[idx]

    # Data augmentation while training.
    if self.train:
        img, bboxes = self.random_flip(img, bboxes)
        img, bboxes, labels = self.random_crop(img, bboxes, labels)

    # Scale bbox locaitons to [0,1].
    w,h = img.size
    bboxes /= torch.Tensor([w,h,w,h]).expand_as(bboxes)

    input_size = 416
    # input_size = random.choice(self.input_sizes)
    img = img.resize((input_size,input_size))
    img = self.transform(img)

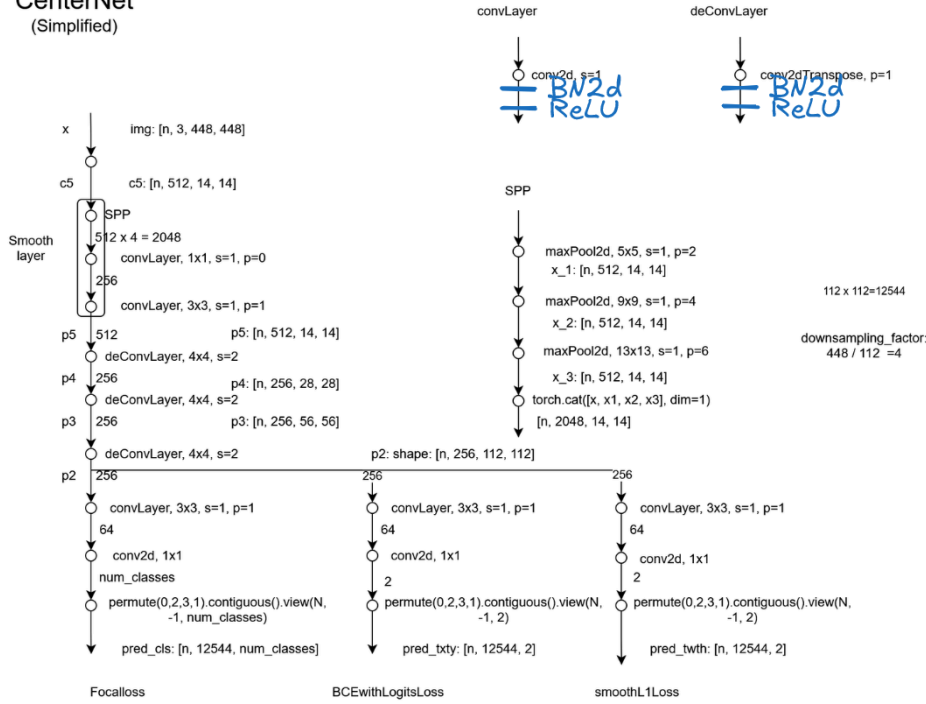
    # Encode data.
    loc_targets, cls_targets, box_targets = self.data_encoder.encode(bboxes, labels, input_size)
    return img, loc_targets, cls_targets, box_targets

### rest of the code is not displayed here

```

5. CenterNet

CenterNet (Simplified)



1.

2. Objects as points.

3. Starting with image $I \in \mathbb{R}^{W \times H \times 3}$

4. Training for keypoint classification:

1. The network will produce a keypoint **heatmap** $\hat{Y} \in [0, 1]^{\frac{W}{R} \times \frac{H}{R} \times C}$

1. R is the output stride, imagine the total effect of all the strides in the convolution operations until the last feature map as output.

2. C is the no. of keypoint types, for example:

1. C=17 in human pose estimation

2. C=80 object categories in object detection (COCO)

3. use R=4, default in literature. R is **downsampling stride**. Image is **downsampled** R times in resolution.

2. To construct the target $Y \in [0, 1]^{\frac{W}{R} \times \frac{H}{R} \times C}$,

1. Let the ground truth keypoint in image pixel coordinate be $p = (p_x, p_y)$. Then compute $\tilde{p}$ the lower resolution equivalent of p as:

1. For example image is of size 28x28x3 divided by 4x4 tiles to become 7x7 grid. Say $p=(26,27)$, $R=4$,

$$1. \tilde{p} = \frac{p}{R} = \left(\frac{26}{4}, \frac{27}{4}\right) = (6.5, 6.75), \lfloor \frac{p}{R} \rfloor = (6, 6),$$

$$2. \tilde{p} = \frac{p}{R} = \left(\frac{25}{4}, \frac{24}{4}\right) = (6.25, 6.), \lfloor \frac{p}{R} \rfloor = (6, 6).$$

2. (x, y) being the coordinate of Y .

3. Imagine point $(\tilde{p}_x, \tilde{p}_y)$ in the center and 8 points (x_i, y_i) surrounding it.

4. Then for each class c , splat the c th dimension of Y with:

$$5. Y_{xyc} = Y(x, y, c) = \exp\left(-\frac{(x-\tilde{p}_x)^2 + (y-\tilde{p}_y)^2}{2\sigma_p}\right).$$

6. If two gaussians of the **same** class overlap, we take the element wise maximum.

7. The training objective is pixel wise logistic regression with focal loss:

$$8. L_k = \sum_{xyc} -\frac{1}{N} \begin{cases} (1 - \hat{Y}_{xyc})^\alpha \log(\hat{Y}_{xyc}) & \text{if } Y_{xyc} = 1 \\ (1 - Y_{xyc})^\beta (\hat{Y}_{xyc})^\alpha \\ \log(1 - \hat{Y}_{xyc}) & \text{, otherwise} \end{cases}$$

9. Note: notice the pattern for positive class: $(1-p)*\log(p)$, for negative class: $\text{weight}*p*\log(1-p)$.

5. Training for keypoint localisation:

1. The heatmap predicts both the class (there is one heatmap $\frac{H}{R} \times \frac{W}{R}$ predicted for each class c) and the centers of each bounding box is the 'center' of each Gaussian splatted on the heatmap. However, the centers of the heatmap grid are integers. Therefore, there needs to be another head to predict the offsets so that final bbox predictions have decimal values or as floats.

2.

$$L_{off} = \text{localisation error in x coordinate} + \text{localisation error in y coordinate}$$

$$3. \quad L_{off} = \frac{1}{N} \left(\sum_{\hat{p}_x} |\hat{O}_{\hat{p}_x} - (\frac{p_x}{R} - \lfloor \frac{p_x}{R} \rfloor)| + \sum_{\hat{p}_y} |\hat{O}_{\hat{p}_y} - (\frac{p_y}{R} - \lfloor \frac{p_y}{R} \rfloor)| \right)$$

6. Training to recover width and height:

1. k th Keypoint: $p_k = (p_x, p_y)$ with bbox: (x_1, y_1, x_2, y_2) , $p_k = (\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2})$
2. $\hat{S}_k$: predicted bbox width & height, $(\hat{x}_2^{(k)} - \hat{x}_1^{(k)}, \hat{y}_2^{(k)} - \hat{y}_1^{(k)})$
3. S_k : ground truth width & height, $(x_2 - x_1, y_2 - y_1)$
4. Without normalizing the scale and directly use raw pixel coordinates.

$$5. \quad L_{size} = \frac{1}{N} \sum_{p_k} |\hat{S}_{p_k} - S_k|$$

7. Then the overall training loss:

$$1. \quad L = L_k + \lambda_{size} L_{size} + \lambda_{off} L_{off}$$

2. we set $\lambda_{size} = 0.1$ and $\lambda_{off} = 1$, means penalize object center offset more than fitting object width & height.

8. The final output size is $(\frac{W}{R}, \frac{H}{R}, C + 4)$, 4=2+2, 2 values for bbox width and height, and 2 values for the centers p_x and p_y offsets.

9. Code: get maximum scores from a 4D heatmap: topk.py

```
# -*- coding: utf-8 -*-
"""
Created on Thu Jun 18 21:08:57 2026

@author: reina
"""

import torch

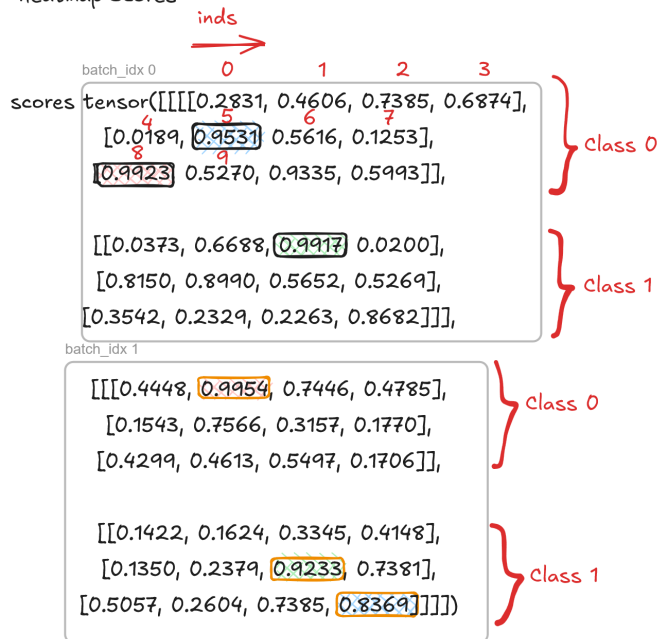
def gather_feat(feats, inds):
    print('feats.shape', feats.shape) # (N, C*k_whc, 1)
    print('inds.shape', inds.shape) # (N, k_whc)
    dim = feats.shape[2]
    inds = inds.unsqueeze(2).expand(inds.shape[0], inds.shape[1], dim)
    return feats.gather(dim=2, index=inds)

if __name__ == '__main__':
    N, C, H, W = 2, 2, 3, 4 # Heatmap shape
    scores = torch.rand(N, C, H, W) # Heatmap scores
    k = 3
    topk_scores, topk_inds = torch.topk(scores.view(N, C, -1), k=k, dim=-1)
    # Top values in scores across spatial dimensions
    print('topk_scores.shape', topk_scores.shape) # (N, C, k_wh)
    print('topk_inds.shape', topk_inds.shape) # (N, C, k_wh)
    print(type(topk_scores))
    print(type(topk_inds))
    topk_inds = topk_inds % (H*W) # Take modulo
    topk_score, topk_ind = torch.topk(topk_scores.view(N, -1), k=k, dim=-1)
    # Top values across the channel dimension
    print('topk_score.shape', topk_score.shape) # (N, k_whc)
    print('topk_ind.shape', topk_ind.shape) # (N, k_whc)
    print(type(topk_score))
    print(type(topk_ind))

    topk_inds = gather_feat(topk_inds.view(N, -1, 1), topk_ind.view(N, k))
    print('topk_inds.shape', topk_inds.shape)

    topk_cls = torch.floor_divide(topk_ind, k).int()
    print('\n')
    print('scores', scores)
    print('\n')
    print('topk_score', topk_score)
    print('topk_inds', topk_inds)
    print('topk_cls', topk_cls)
```

Heatmap Scores



topk_score tensor([[[0.9923, 0.9917, 0.9531],
[0.9954, 0.9233, 0.8369]])

topk_inds tensor([[[8],
[2],
[5]],

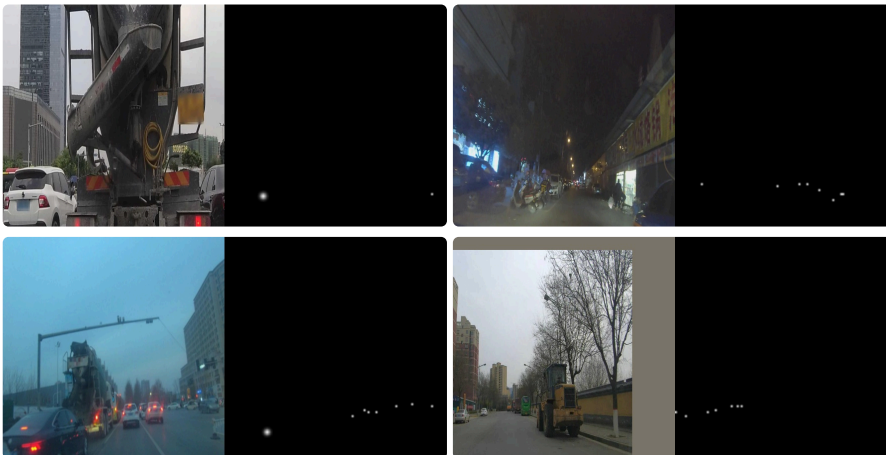
[[1],
[6],
[11]]])

topk_cls tensor([[[0, 1, 0],
[0, 1, 1]], dtype=torch.int32)

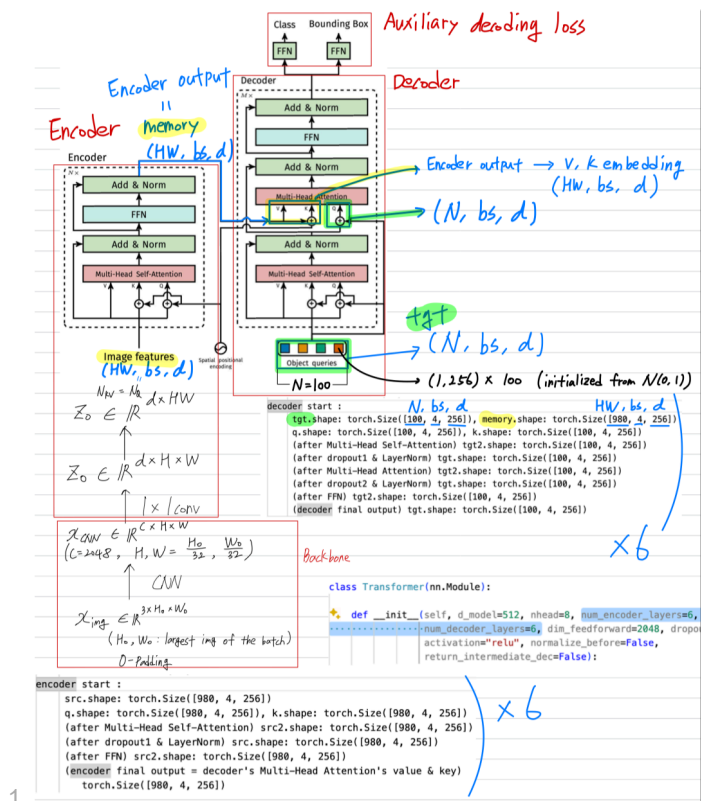
10.

apply topk and gather to heatmap to get values and indices of max scores

11.



6. DETR



1. Traditional methods solve in an indirect way

- Anchor-based: solve surrogate regression and classification problems.
- Their performances are significantly influenced by postprocessing steps to collapse near-duplicate predictions, by design of the anchor.
- DETR is a direct **set prediction** approach to bypass the surrogate tasks, so-called end-to-end.
 - Given an image, the model must predict **an unordered set of all the objects present**, each represented by its class, along with a tight bounding box surrounding each one.
 - DETR predicts in parallel the final set of detections.
 - The CNN is used for feature learning, the transformer is used to make predictions.
 - This formulation is particularly suitable for transformers.
 - Using transformers and parallel decoding.
- Compared to YOLO approach:
 - Object query in place of anchor boxes.
 - During training Hungarian matching in place of 'NMS' like bounding box assignment.
 - As a result DETR is NMS free (no need of NMS during training or inference).
- Eliminates duplicate predictions.
- multi-label classification:
 - multiclass: dog: 0.9 -- uses Softmax activation function or cross entropy loss.
 - multilabel: dog: 0.9, trees: 0.75, -- uses Sigmoid or binary cross entropy loss, set prediction.
 - Code: dummy example of multi-label classification in pytorch.

```

import torch
import torch.nn as nn
import torch.optim as optim

model = nn.Linear(20, 5) # predict logits for 5 classes
x = torch.randn(1, 20) # batch_size = 1
y = torch.tensor([[1., 0., 1., 0., 0.]]) # get classA and classC as active

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.SGD(model.parameters(), lr=1e-1)

for epoch in range(20):
    optimizer.zero_grad()
    output = model(x)
    print('output.shape', output.shape) # (batch_size, n_classes): (1, 5)
    loss = criterion(output, y)
    loss.backward()
    optimizer.step()
    print('Loss: {:.3f}'.format(loss.item()))

```

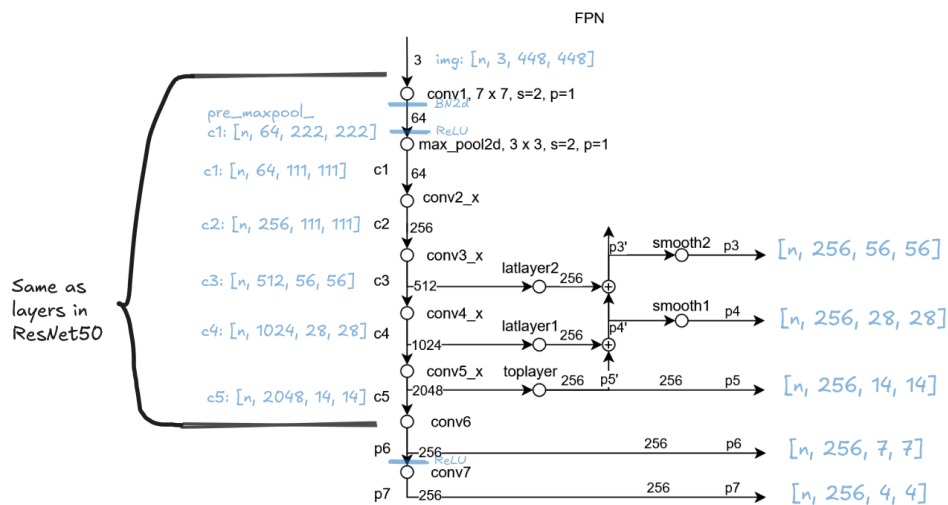
4. Issue with multilabel classification is when the model predicts the same amount of probability for two different objects. In object detection, if the region proposals are too near to each other.
5. In the context of object detection, DETR predicts N number of object boxes and N is much larger than the amount of ground truth bounding boxes G.
7. Bipartite matching is used to match the set of detections N to set of ground truth boxes in G. Note: Bipartite matching is the process of pairing vertices from two sets in such a way that each vertex is paired with at most one vertex from the other set, and the total number of paired vertices is maximized.
8. DETR infers a fixed-size set of N predictions, where N is set to be significantly large than the typical number of objects in an image.
9. Let us denote by y the ground truth set of objects, and $\hat{y} = \{\hat{y}_i\}_{i=1}^N$ the set of N predictions.
10. y also as a set of size N padded with $\emptyset$ (no object).
11. To find a bipartite matching between these two sets we search for a permutation of N elements $\sigma \in \Gamma_N$ with the lowest cost:
 1. $\hat{\sigma} = \arg \min_{\sigma \in \Gamma_N} \sum_i^N L_{match}(y_i, \hat{y}_{\sigma(i)})$
12. Components:
 1. A CNN backbone to extract a compact feature representation.
 2. An encoder-decoder transformer.
 3. A simple feed forward network (FFN) that makes the final detection prediction.
13. DETR architecture
 1. Backbone
 1. Starting from the initial image $x_{img} \in \mathbb{R}^{3 \times H_0 \times W_0}$, CNN backbone generates a lower resolution activation $\mathbb{R}^{C \times H \times W}$, typically $C = 2048$ and $H, W = H_0/32, W_0/32$.
 2. Transformer encoder
 1. First, a 1×1 convolution reduces the channel dimension from C to a smaller dimension d , creating a new feature map $z_0 \in \mathbb{R}^{d \times H \times W}$.
 2. Then, we collapse the spatial dimensions of z_0 into one dimension, resulting in a $d \times HW$ feature map (because the encoder expects tokens/sequence as input).
 3. Besides, we need to supplement it with positional encodings.
 4. Now we can feed it into encoder.
 3. Transformer decoder
 1. Difference is that DETR decodes the N objects in parallel at each decoder layer.
 2. Original transformer decoder uses AR model that predicts the output sequence one element at a time.
 3. Like the encoder, needs to add learnt positional encodings, which is called **object queries**.
 4. Prediction feed-forward networks
 1. A 3-layer perceptron with ReLU activation function and hidden dimension d , and a linear projection layer.
 2. Predicts the normalized center coordinates, height and width of the box and the class label.
14. DETR loss function
 1. From the architecture, we know DETR infers a fixed-size set of N predictions in single pass.
 1. So, how to assess the quality of N predictions?
 2. DETR does bipartite matching between predicted and ground truth objects, and calculates loss based on matching result.
 3. Matching cost:
 1. c_i is the target class label
 2. $\sigma(i)$ is the index of prediction that matches i^{th} target.
 3. $b_i \in [0, 1]^4$ is a vector that defines ground truth box center coordinates and its relative height and width.
 4. Matching cost:

Method	Matching Complexity	Memory Usage	Training Convergence	Inference Speed
DETR + Hungarian	$O(N^3)$ per image	High (transformer attention)	slow (500+ epochs)	Moderate
FasterRCNN	$O(N)$ with NMS	Moderate	Fast (12-36 epochs)	Fast
YOLO series	$O(N \log N)$ with NMS	Low	Fast (100-300 epochs)	Very fast

5. SPP

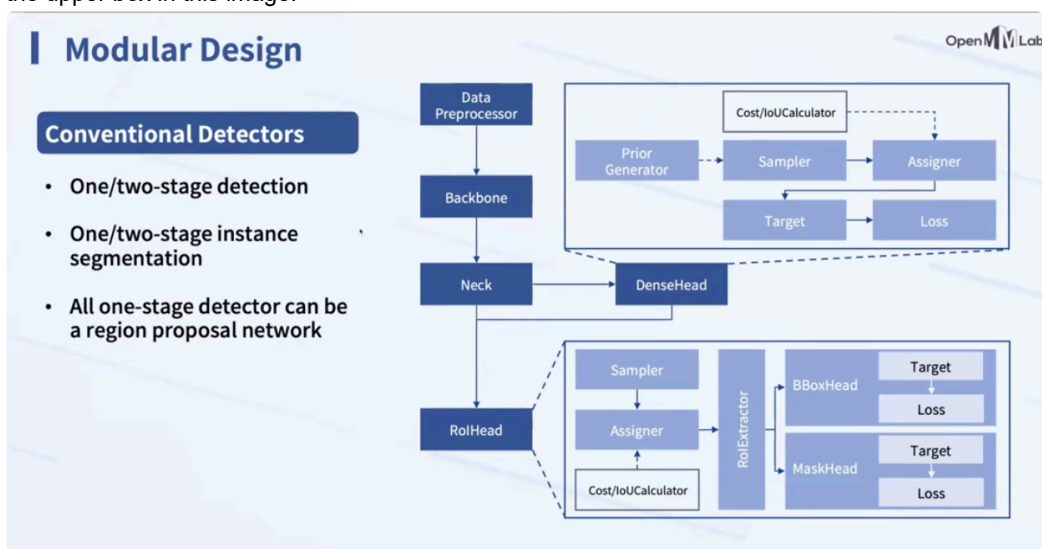
1. Used to adaptively produce feature maps of constant sizes, by recomputing the necessary kernel sizes and strides.

6. FPN

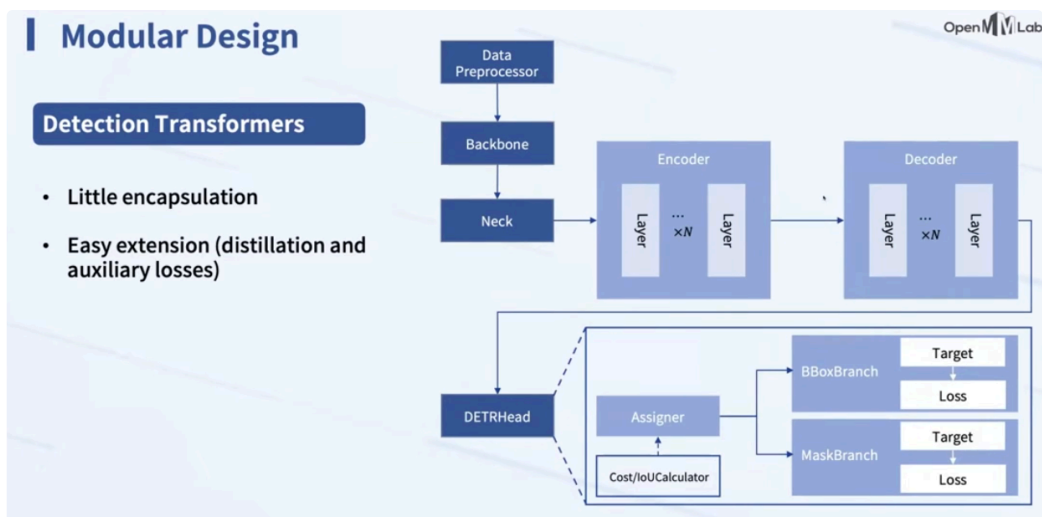


- 1.
2. used in detectron2 as backbone for base RCNN (Region CNN).
3. Used to perform multi-scale object prediction.

mm detection2d :One-stage detectors can be viewed as a Region Proposal Network in a two-stage network, which is the upper box in this image.



DETR



3D

References:

- ## 1. Multi-label classification

2. MSDN, [github](#)
3. <https://erdem.pl/2020/02/understanding-region-of-interest-ro-i-pooling>
4. <https://erdem.pl/2020/02/understanding-region-of-interest-part-2-ro-i-align>
5. YOLOv1, <https://www.slideshare.net/slideshow/yolo-v1-241861792/241861792#17>
6. Faster R-CNN, <https://www.slideshare.net/slideshow/pr12-faster-rcnn170528/78497784> by Jinwon Lee
7. DETR, <https://www.slideshare.net/slideshow/pr284-endtoend-object-detection-with-transformersdetr/239265010> by Jinwon Lee
8. <https://blog.ononoki.org/detr-introduction/>
9. <https://mangohost.net/blog/introduction-to-detr-hungarian-algorithm-part-1/>